SMART ITINERARY PLANNER

AI-Based Travel Planning System

B. Tech. Computer Science & Engineering

CSD334 MINI PROJECT 2026

23CSA49 MDL23CS145 PRADHAM L RAJ

23CSA57 MDL23CS163 SAQUIB RIZWAN A S

23CSA65 MDL23CS191 VIGNESH ANUP

Govt. Model Engineering College, Kochi

Department of Computer Engineering

Model Engineering College, Ernakulam

Thrikkakara, Kochi 682021

Phone: +91.484.2575370

http://www.mec.ac.in

hodcs@mec.ac.in

Model Engineering College, Ernakulam
Dept. of Computer Engineering

Govt. Model Engineering College, Kochi

C E R T I F I C A T E

This is to certify that, this report titled ***SMART ITINERARY PLANNER Project Report*** is a bonafide record of the work done by

23CSA49	**MDL23CS145**	**Pradham L Raj**
23CSA57	**MDL23CS163**	**Saquib Rizwan A S**
23CSA65	**MDL23CS191**	**Vignesh Anup**

Sixth Semester B. Tech. Computer Science & Engineering students, for the course work in **CSD 334 Mini Project** which is developed for Mini Project Work, under our guidance and supervision, in partial fulfillment of the requirements for the award of the degree, B. Tech. Computer Science and Engineering of **APJ Abdul Kalam Technological University**.

Guide

Ms. Aysha Fymin Majeed
Associate Professor
Department of Computer Engineering

Coordinator

Ms. Athira S Nair
Assistant Professor
Department of Computer Engineering

Head of the Department

March 12, 2026

Dr. Binu VP
Associate Professor
Department of Computer Engineering

Acknowledgement

We would like to express our sincere gratitude to Dr. Jayachandran E S, Principal, Government Model Engineering College, and Dr. Binu V P, Head of the Department of Computer Engineering, for providing us with the necessary infrastructure and opportunities to carry out this project.

We are deeply thankful to our project guide, **Ms. Aysha Fymin Majeed**, Associate Professor, Department of Computer Engineering, for her invaluable guidance, constant motivation, and continuous support throughout the duration of this mini project. Her expertise and feedback have been instrumental in the successful completion of this endeavor.

We also extend our sincere thanks to our coordinator, **Ms. Athira S Nair**, Assistant Professor, Department of Computer Engineering, for her coordination and support. Furthermore, we are grateful to all the faculty members of the Department of Computer Engineering who have directly or indirectly contributed to the realization of our project goals. Finally, we thank our peers, friends, and family for their unwavering encouragement.

Pradham L Raj

Saquib Rizwan A S

Vignesh Anup

Abstract

The rapid growth of the travel and tourism industry has led to an increasing demand for intelligent and personalized travel planning tools. Traditional methods of manually planning itineraries and seeking accommodations are often tedious, time-consuming, and fail to adapt to dynamic geographic constraints or user-specific preferences. This project proposes a comprehensive **Smart Itinerary Planner**, an AI-based system designed to streamline travel scheduling through multi-destination itinerary generation and user preference modeling.

A core innovation of this project is the integration of an intelligent interest-based tagging system, enabling precise categorization and retrieval of travel destinations. Additionally, the system incorporates a specialized Hotel Recommendation System as an independent add-on module to the core itinerary planner. Without altering the underlying itinerary generation logic, this module recommends hotels, estimates star ratings through analysis of descriptions, reviews, and existing ratings, and estimates per-night pricing tailored specifically to a Kerala-based pricing model.

Furthermore, dynamic hotel planning features are implemented, allowing users to opt for different accommodations for each day. This dynamic allocation is contingent upon the end location of the itinerary day, effectively reducing unnecessary commute times and enriching the overall travel experience. The user maintains the freedom to either continue their previous stay or change hotels independently of the generated itinerary structure. Ultimately, the integration of these features culminates in an intelligent, robust, and user-centric platform that simplifies complexities associated with travel organization.

Contents

Acknowledgement **i**

Abstract **ii**

1 Introduction **1**

1.1 Introduction 1

1.2 Literature Survey 1

1.2.1 Satisfaction-Based Itinerary Planning 2

1.2.2 Interaction-Based Itinerary Recommendation 2

1.2.3 +Tour: Smart Tourism with Multi-access Edge Computing . 3

1.2.4 Intention-Aware Itinerary Recommendation 3

1.2.5 ITINERA: LLM-Based Itinerary Planning 4

1.2.6 Learning-Based Itinerary Optimization 4

1.3 Summary of Literature Survey 5

1.4 Proposed Project 6

1.4.1 Problem Statement 6

1.4.2 Proposed Solution 6

2 Report of Preparatory Work **8**

2.1 Comparative Analysis 8

2.2 System Study Report 9

2.2.1 Feasibility Study 9

2.2.2 Existing System 9

2.2.3 Proposed System 10

3 Software Requirements Specification **11**

3.1 Introduction 11

3.2 Functional Requirements 11

3.3 Non-Functional Requirements 12

3.4 Hardware Requirements 13
3.5 Software Requirements 13

4 Software Design Description 14

4.1 Introduction 14
4.2 System Overview 14
4.3 Functional Description 15
4.4 System Design 15
4.5 Implementation Details 16
4.6 Testing and Quality Assurance 16
4.7 UML Diagrams 17
4.7.1 Use Case Diagram 17
4.7.2 Architecture Diagram 18
4.7.3 State Diagram 18
4.7.4 Level 0 Data Flow Diagram 20
4.7.5 Level 1 Data Flow Diagram 21
4.7.6 Activity Diagram 22

5 Output 23

5.1 Implementation 23
5.1.1 Technology Stack 23
5.1.2 Core Itinerary Processing 23
5.1.3 The Hotel Suggestion Add-on 24
5.1.4 Rating & Pricing Models 24
5.1.5 Authentication and Security 25
5.1.6 Workflow 25
5.2 Testing 25
5.2.1 Unit Testing 26
5.2.2 Integration Testing 26
5.2.3 Functional Testing 26
5.2.4 System and User Acceptance Testing 27
5.2.5 Limitations in Testing 27

5.3 Screenshots 27

6 Conclusion **30**

6.1 Future Scope 30

6.2 Conclusion 31

References **32**

Chapter 1

Introduction

1.1 Introduction

Travel and tourism constitute a significant segment of the global economy, providing vital cultural exchange and economic value. With the modern traveler desiring more distinct, personalized, and seamless travel experiences, the complexity of efficiently planning a trip has surged. Planning an optimal itinerary requires navigating an overwhelming amount of information concerning tourist spots, travel times, accommodation availability, and estimated costs. Travelers often find themselves switching between multiple platforms to piece together a coherent travel plan, mapping out the routes, and manually trying to minimize logistical inefficiencies.

To address these pain points, the **Smart Itinerary Planner** proposes an automated, AI-driven environment. The system models user preferences to generate robust multi-destination itineraries that intuitively understand where a traveler wants to go based on smart, interest-based tagging of destinations. This dynamic application shifts the burden of logistical planning from the user to the underlying algorithms, saving significant time and reducing the cognitive load necessary to schedule vacations or professional trips effectively. The platform distinguishes itself through modular add-ons to improve ease of use and recommendation precision further, setting a strong foundational framework in automated travel solutions.

1.2 Literature Survey

In constructing an intelligent itinerary generation and hotel recommendation engine, it is necessary to study various approaches suggested by researchers and the current industry standards over recent years.

1.2.1 Satisfaction-Based Itinerary Planning

Liu et al. (2024) propose a satisfaction-based itinerary planning framework that models tourist experience as a multi-factor optimization problem. The study presents a comprehensive approach to travel planning that goes beyond simple routing algorithms.

Methodology: Unlike traditional shortest-path routing, the system considers attraction popularity, diversity, time utilization, dining preferences, accommodation constraints, and tourist fatigue. Constraint satisfaction algorithms are used to balance these competing factors in real time.

Advantages: The generated itineraries are realistic and align closely with human travel behavior, offering a more personalized experience than static planning tools.

Disadvantages: The inclusion of multiple constraints increases computational complexity, making scalability challenging for large urban environments with hundreds of points of interest.

1.2.2 Interaction-Based Itinerary Recommendation

Otaki and Baba (2025) introduce an interaction-based itinerary recommendation system that enables users to modify itineraries through an interactive map interface. This paper focuses on closing the feedback loop between users and the recommendation engine.

Methodology: User interactions are treated as implicit feedback for refining ranking models. Machine learning techniques are applied to continuously update the recommendation model based on user behavior patterns.

Advantages: This approach reduces data sparsity and improves personalization by learning directly from user behavior, leading to increasingly accurate recommendations over time.

Disadvantages: Despite improved adaptability, the system requires continuous user interaction, which may not be suitable for all users, particularly those seeking a one-click planning experience.

1.2.3 +Tour: Smart Tourism with Multi-access Edge Computing

Espera et al. (2025) present +Tour, a smart tourism platform that jointly optimizes itinerary planning and network resource allocation using Multi-access Edge Computing (MEC). The paper targets next-generation smart tourism use cases.

Methodology: The system supports augmented reality and multimedia services while ensuring low-latency responses through strategically placed edge computing nodes near tourist destinations.

Advantages: The combination of itinerary planning with network optimization allows enriched experiences including real-time AR overlays at tourist sites, offering a high level of digital engagement.

Disadvantages: The main drawback of this approach is its dependency on infrastructure-heavy deployments, making it unsuitable for regions with limited edge computing infrastructure.

1.2.4 Intention-Aware Itinerary Recommendation

Cui et al. (2025) propose an intention-aware itinerary recommendation framework that captures dynamic user intent using deep learning techniques. The paper addresses the challenge of modeling users whose preferences evolve throughout the planning process.

Methodology: Transformers are used to model temporal user behavior, while Graph Convolutional Networks (GCNs) learn relationships between points of interest. This joint modeling enables simultaneous POI selection and route planning.

Advantages: The dual-model architecture allows the system to simultaneously optimize for user preference and spatial route efficiency, yielding highly contextual recommendations.

Disadvantages: The system suffers from high model complexity and computational overhead, requiring significant hardware resources for both training and real-time inference.

1.2.5 ITINERA: LLM-Based Itinerary Planning

Tang et al. (2025) introduce ITINERA, a large language model-based itinerary planning system that supports natural language travel queries. The paper explores how conversational AI can be applied to complex spatial planning tasks.

Methodology: The system performs preference-aware POI retrieval and cluster-based spatial optimization to generate personalized itineraries from free-form natural language inputs provided by users.

Advantages: The natural language interface dramatically lowers the barrier to entry, allowing users to describe their travel goals conversationally rather than through structured forms.

Disadvantages: The reliability of the system depends heavily on the accuracy of the underlying language model, and errors in natural language understanding can propagate to produce suboptimal itineraries.

1.2.6 Learning-Based Itinerary Optimization

Morris and Petrova (2024) propose a learning-based itinerary optimization framework that integrates graph-based multi-stop routing with reinforcement learning. The paper frames itinerary construction as a sequential decision-making problem.

Methodology: The system dynamically evaluates routes based on distance, time, and recommendation relevance using a reinforcement learning agent trained on historical travel data.

Advantages: The reinforcement learning approach allows the system to improve over time without requiring explicit programming of optimization rules, leading to emergent routing strategies.

Disadvantages: The approach requires extensive training data and computational resources, and its performance is highly sensitive to the quality and representativeness of the training dataset.

1.3 Summary of Literature Survey

The following table summarizes the key papers reviewed, their focus areas, methodologies, and primary limitations.

Table 1.1: Summary of Literature Survey

Sl.	**Title**	**Year**	**Methodology**	**Key Limitation**
1	Satisfaction-Based Itinerary Planning	2024	Multi-factor constraint satisfaction	High computational complexity
2	Interaction-Based Itinerary Recommendation	2025	Implicit feedback ranking	Requires continuous interaction
3	+Tour: Smart Tourism with MEC	2025	Edge-computing optimization	Infrastructure-heavy deployment
4	Intention-Aware Itinerary Recommendation	2025	Transformer + GCN joint model	High computational overhead
5	ITINERA: LLM-Based Itinerary Planning	2025	Natural language POI retrieval	LLM accuracy dependency
6	Learning-Based Itinerary Optimization	2024	Reinforcement learning routing	Requires extensive training data

1.4 Proposed Project

1.4.1 Problem Statement

Manual travel planning is fraught with friction. Users spend extensive hours researching locations, mapping distances, and determining suitable accommodations. Even when using modern web applications, generated itineraries are often rigid.

Furthermore, recommending accommodations typically involves localized spatial searches that lack deep integration with the traveler's dynamically changing daily endpoints. When travelers visit multiple diverse locations over a span of several days, returning to the same base hotel can cause severe retracing of routes, increasing travel time and expenses.

There is an operational deficit in estimating reliable real-time metadata — such as true star ratings or localized pricing — for hotels, particularly in specific regional markets like Kerala, where conventional overarching algorithms often fail to capture localized economic variations. Additionally, modifying hotel plans should ideally not disrupt or necessitate the recalculation of the carefully ordered travel itinerary itself. The lack of modularity in existing systems results in a fragile user experience where changes to accommodation break the generated daily plans.

1.4.2 Proposed Solution

The proposed solution is an AI-based Smart Itinerary Planner capable of constructing customized, multi-destination plans centered around dynamic user preference modeling. The system integrates advanced interest-based tagging of places to filter and recommend the optimal array of locations tailored to the specific user's tastes.

Critically, to solve the accommodation logistics issues, the system introduces an independently functioning Hotel Recommendation System as a modular add-on. The core itinerary generation logic remains untouched while this add-on layers accommodation intelligence over the generated plan. Key features of this solution include:

- **Hotel Recommendation System:** Operates contextually, using the terminal location of a user's daily journey to suggest relevant hotels, thereby drastically reducing unnecessary commuter travel.
- **Dynamic Hotel Planning:** Allows users to select different accommodations for each individual day of the trip. Users are provided options to either check into a new hotel perfectly situated near their day's final attraction, or flexibly opt to continue their previous stay.
- **AI-based Rating Estimation:** Extracts contextual meaning from ratings, raw descriptions, and aggregated reviews to predict and assign accurate hotel star ratings.
- **Regional Pricing Model:** Implements a specialized Kerala-based pricing estimation engine, providing realistic daily hotel budget expectations.

Chapter 2

Report of Preparatory Work

2.1 Comparative Analysis

The following table highlights the functional disparities between existing prominent systems and the proposed Smart Itinerary Planner.

Table 2.1: Features Comparison with Existing Systems

Features	**Google Travel**	**TripAdvisor App**	**Proposed System**
Multi-destination Routing	Basic / Static	Moderate	High / AI-Optimized
Interest-Based POI Tagging	Yes	Yes	Yes (Contextually Weighted)
End-of-day Dynamic Hotels	No (Assumes Central Base)	Manual user selection	Yes (Automated via terminal POI)
Review-based Star Estimation	Relies on predefined stars	Extracted from user data	Advanced NLP Estimation
Localized Cost Model	Generalized global metrics	Generalized	Specialized Kerala-based pricing
Modularity of Bookings	Monolithic	Disjointed	Highly Modular add-on logic

2.2 System Study Report

2.2.1 Feasibility Study

A system's viability is fundamentally measured on technical, economic, and behavioral paradigms.

Technical Feasibility: The project requires server architecture capable of running geographic optimization algorithms and basic machine-learning inference for rating estimation. Leveraging modern open-source web frameworks such as React/Vue for frontend and Node.js/Python for backend, combined with robust cloud mapping APIs (MapBox or Google Maps SDK), makes the technical realization straightforward and highly scalable.

Economic Feasibility: As a software-based service primarily relying on free-tier APIs during development and cloud computing platforms such as AWS or Heroku with pay-as-you-go structures, the initial capital required is minimal. Long-term commercialization could be driven by hotel affiliate links or localized tourism boards, making the operational costs highly sustainable.

Behavioral (Operational) Feasibility: Travelers increasingly expect intelligent guidance. Designing an intuitive User Interface (UI) where the complexities of itinerary creation and destination matching are hidden behind a single click guarantees high user acceptance. The add-on nature of the hotel module ensures users who prefer only itinerary planning are not forced into unwanted flows, representing flexible user control.

2.2.2 Existing System

In defining the existing solutions such as standard travel portals or manual agency workflows, several systemic flaws form bottlenecks in the travel experience:

- Static itineraries where any localized delay renders the rest of the plan inaccurate.
- Hotel recommendations default to a city's center rather than the traveler's

dynamic location at the end of the day.

- Pricing data across aggregators is often skewed by global algorithms, hiding realistic, regional pricing.
- Inflexible daily continuation; travelers are restricted to checking out or returning to the same base, lacking automated mid-trip transit suggestions.

2.2.3 Proposed System

The proposed Smart Itinerary Planner addresses these issues explicitly through:

- Intelligent tagging of Points of Interest based on user profile inputs.
- Algorithm-driven daily plans that conclude precisely at a point, followed instantly by hotel recommendations local to that endpoint.
- A distinct, decoupled add-on engine for the hospitality layer, ensuring core geographical routing is generated and preserved perfectly.
- Option to automatically reserve multi-stay plans across different regions per day to minimize commute, alongside localized price prediction functions.

Our system in comparison to existing systems offers:

- Centralized AI-powered itinerary generation with interest-based POI tagging.
- Dynamic, location-aware hotel recommendations at the end of each travel day.
- Modular, decoupled hotel add-on that does not interfere with core routing logic.
- Specialized Kerala-based pricing model for accurate regional cost estimation.
- AI-inferred hotel star ratings derived from review sentiment and amenity keywords.

Chapter 3

Software Requirements Specification

3.1 Introduction

This chapter defines the technical specifications required to bring the AI-based Smart Itinerary Planner to production. It breaks down what the system must do computationally and lists out the conditions under which it must operate.

3.2 Functional Requirements

Functional requirements describe the core features and operations that the Smart Itinerary Planner must support. These requirements define how users interact with the system and how the system responds to user actions.

1. **User Preference Engine:** The system shall allow users to register and define specific interests such as historical sites, adventure activities, culinary experiences, and relaxation destinations.

2. **Itinerary Generation Engine:** The system shall accurately create a multi-day trip route connecting diverse places derived from semantic interest-based tags, optimizing for travel time and user satisfaction.

3. **Add-on Module – Hotel Recommendation:** Upon generation of a daily itinerary, the system shall invoke an independent plugin capable of overlaying nearby accommodations against the last destination of the day.

4. **Dynamic Day-to-Day Hotel Booking:** The system must provide UI toggles enabling the user to choose to "Stay here tonight" or "Continue previous stay", generating dynamic pricing based on these choices.

5. **Star Rating Estimator:** The system shall process hotel metadata including reviews and descriptions to generate an AI-estimated 1–5 star hotel quality metric.

6. **Kerala-based Price Computation:** The platform must dynamically adjust price estimates for hotels leveraging a specialized econometric model tuned specifically for the State of Kerala.

7. **Administrative Management:** The system shall provide administrators with tools to manage POI databases, update hotel listings, and monitor system performance.

3.3 Non-Functional Requirements

The system shall satisfy the following non-functional requirements:

1. **Performance:** The core itinerary algorithm must resolve and display multi-day geographic routes within 5 seconds of the request.

2. **Modularity:** The Hotel system must be strictly architected as a decoupled microservice or separate modular class. Failures or changes in hotel selection must not trigger a recalculation of the itinerary.

3. **Scalability:** The backend must handle increasing concurrency, notably multiple users running complex routing inferences simultaneously without degradation in response time.

4. **Usability:** The user interface should remain accessible and responsive on mobile and desktop devices, keeping the learning curve flat for non-technical users.

5. **Reliability:** API endpoints connecting to mapping servers and regional hotel databases should implement robust fallback mechanisms and retry logic to maintain uptime.

6. **Security:** User data, preference profiles, and travel plans must be protected using secure authentication and access control mechanisms.

3.4 Hardware Requirements

- **Server Side:** A multi-core processor with a minimum of 4GB RAM to handle asynchronous routing and text analysis models. Sufficient storage for hotel databases and user preference data.
- **Client Side:** Any modern smart device (Smartphone/PC) with at least 2GB RAM, a modern web browser, and an operational internet connection.
- **Network Interface:** A reliable network interface with stable internet connectivity is required to support web-based access and communication between users and the server.

3.5 Software Requirements

- **Operating System:** Platform-agnostic (Cloud-deployed on Linux environments such as Ubuntu 20.04 LTS).
- **Frontend:** HTML5, JavaScript (React.js or Next.js), Tailwind CSS for responsive UI design.
- **Backend/API:** Node.js with Express / Python with FastAPI for AI and string processing endpoints.
- **Database:** MongoDB (NoSQL) for storing user preferences, dynamic itineraries, and tagged POIs.
- **AI/ML Add-ons:** Integration of Scikit-learn or similar NLP pipelines to process rating estimations.
- **Mapping APIs:** Google Maps API or MapBox SDK for geographic data and routing.
- **Version Control:** Git and GitHub for source code management.

Chapter 4

Software Design Description

4.1 Introduction

The primary objective of software design is to abstract the complexities of solving spatial routing and predictive analytics into an ordered, modular architecture. The design focuses heavily on guaranteeing the independence of the Hotel System Add-on to meet the critical design constraint of non-interference with the core itinerary logic.

4.2 System Overview

The system operates on an automated, sequential pipeline:

1. User supplies constraints including the number of days, starting location, and interest categories.
2. The core AI planner selects and optimally maps points of interest based on tagging models.
3. The itinerary is finalized and locked as an immutable structure.
4. The isolated hotel add-on fetches the terminal coordinates of each day's plan.
5. Rating processing and pricing algorithms estimate local stays based on the terminal location.
6. The user refines stay preferences without recalculating the core itinerary locations.

4.3 Functional Description

The major functions are grouped into discrete handlers:

- **Auth and Profile Handler:** Manages user authentication tokens, interest profiles, and historical trip preferences.
- **Tagging Logic Engine:** Cross-references the database of Points of Interest (POIs), utilizing a weighting metric against the user's declared interests to fetch the most relevant sights.
- **Core Itinerary Builder:** Calculates shortest paths and reasonable chronological ordering of sightseeing locations using geographical API tools and TSP-inspired algorithms.
- **Hotel Recommendation Add-on Plugin:** Takes an immutable (read-only) feed of generated daily endpoints and fetches, processes, and calculates star ratings and Kerala-based prices to supply accommodation data decoupled from core pathfinding.

4.4 System Design

The system strictly enforces an n-tier architecture incorporating external integrations for maps and machine learning models.

- **Frontend:** Developed using React.js / Next.js to provide a responsive and interactive web interface.
- **Backend:** Implemented using Node.js with Express or Python with FastAPI to create secure and scalable RESTful APIs.
- **Storage:** MongoDB is used as the NoSQL database to store user preferences, POI records, and itinerary snapshots.
- **ML Service:** A separate FastAPI microservice handles NLP-based star rating estimation and regional pricing inference.

The core logic layer houses the geographical mapping arrays, communicating via RESTful APIs with the Hotel Add-On Service. This isolation ensures that even if the Hotel microservice throws an error or requires a restart, the itinerary itself remains intact and securely displayed on the frontend.

4.5 Implementation Details

While the exact intricacies of the source code are not fundamentally required to grasp the system's behavior, the implementation hinges around passing state arrays to the hotel add-on.

Interest-based Tagging: Implemented using inverted index techniques, where multiple tags such as 'hills', 'history', and 'trekking' are assigned vectors. User preference vectors perform dot products against destination vectors, returning highly contextual matches seamlessly.

AI Hotel Star Rating Estimator: The feature processes scraped textual strings. It uses sentiment polarity analysis on previous user reviews coupled with keyword matching on the hotel's description. Keywords such as 'spa', 'valet', and 'swimming pool' mathematically raise the estimated true-star value, presenting users with an *Intelligent Expected Star Rating* rather than unreliable marketing ratings.

Kerala-Based Pricing Model: This utilizes historical data inputs modeling specific location economics, classifying differences between Munnar during peak season versus Kochi during monsoons, enabling accurate prediction mappings unseen in standard global models.

4.6 Testing and Quality Assurance

Testing design requires isolation testing for modules. The itinerary generator is mocked to provide synthetic daily end-points to validate whether the accommodation logic effectively binds correctly to mapping locations and prices without backward cyclic loops. Testing focuses on performance, reliability, and correctness of the modular separation.

4.7 UML Diagrams

Below are the graphical architectural representations outlining data flow and structural integrations within the project.

4.7.1 Use Case Diagram

usecase_diagram.png

Figure 4.1: Use Case Diagram of Smart Itinerary Planner

4.7.2 Architecture Diagram

architecture_diagram.png

Figure 4.2: System Architecture Diagram

4.7.3 State Diagram

state_diagram.png

Figure 4.3: State Diagram

4.7.4 Level 0 Data Flow Diagram

level0_dfd.png

Figure 4.4: Level 0 Data Flow Diagram (Context Diagram): High-level flow between the User, External APIs, and the Smart Planner.

4.7.5 Level 1 Data Flow Diagram

level1_dfd.png

Figure 4.5: Level 1 Data Flow Diagram: Detailed breakdown of the tagging logic, itinerary creation, and modular hotel pricing modules.

4.7.6 Activity Diagram

activity_diagram.png

Figure 4.6: Activity Diagram

Chapter 5

Output

5.1 Implementation

The implementation phase brings the described architectural designs into functional software. This section describes the technical implementation of the system, including the technology stack, system architecture, and the workflow of different user roles.

5.1.1 Technology Stack

The system is developed using modern web technologies to ensure scalability, performance, and maintainability:

- **Frontend:** HTML5, CSS3, JavaScript, React.js (Next.js) with Tailwind CSS
- **Backend:** Node.js with Express.js / Python with FastAPI
- **Database:** MongoDB (NoSQL)
- **ML Service:** Python (Scikit-learn, NLTK/spaCy for NLP)
- **Mapping APIs:** Google Maps API (Distance Matrix, Places, Geocoding)
- **Version Control:** Git and GitHub

5.1.2 Core Itinerary Processing

The primary objective during execution was to secure the itinerary creation logic strictly as a monolithic pure function, ensuring no mutations occur downstream. The generation starts with constructing nodes of selected destinations. Utilizing routing algorithms akin to the Traveling Salesperson Problem (TSP) constraints,

the system chronologically sorts activities accounting for average transit times and viewing durations. Consequently, each day operates intrinsically with an absolute "start node" and absolute "end node".

5.1.3 The Hotel Suggestion Add-on

As emphasized, the implementation of accommodations occurred exclusively as a subsequent operation overlaying the itinerary structure.

- **Terminal Fetching:** Once an itinerary day is locked, the Add-on reads the geographic coordinates of the day's final POI.
- **Radial Search Strategy:** The system enacts a radius-bound geographical database query searching for hotel entries nearest to this specific terminal POI, dramatically alleviating the inherent user burden of having to calculate the commute from a tourist spot to a prospective hotel.
- **Continuity Controls:** Built into the data payload returned to the client frontend is a logical hook determining whether the terminal location of Day X is geographically close to the terminal location of Day X-1. If they fall within a reasonable distance threshold, the UI prominently triggers an option allowing the user to seamlessly "Continue previous stay", eliminating redundant hotel transitions.

5.1.4 Rating & Pricing Models

Generating realistic metadata for available stay options required specialized logical pipelines.

Star Estimation Pipeline: Hotels lack standardized global evaluations. During the scraping and ingestion phase of hotel data, an NLP inference block analyzes the text. If reviews possess primarily positive sentimental polarity and description matrices feature high-value amenity terms, an algorithmic formula averages these vectors against standard baseline ratings, presenting users with an *Intelligent Expected Star Rating* rather than unreliable marketing ratings.

Pricing Engine Implementation: Developed using historical pricing brackets contextualized to Kerala's tourism zones. The engine accounts for whether the localized endpoint is classified as a tier-1 metropolitan area (like Kochi) or a tier-2 holiday hill station (like Munnar), mathematically scaling the estimated nightly price accordingly.

5.1.5 Authentication and Security

The system ensures secure access using standard web authentication mechanisms. Users access the platform with authenticated credentials, and role-based access control restricts unauthorized actions. Sensitive preference data is protected using backend validation and secure API communication.

5.1.6 Workflow

The workflow of the system is as follows:

1. User registers, sets travel interests and trip constraints.
2. The tagging engine selects and ranks relevant POIs.
3. The routing algorithm generates an optimized multi-day itinerary.
4. The hotel add-on fetches end-of-day coordinates and recommends nearby accommodations.
5. The user reviews the itinerary and modifies hotel selections without disrupting the route.
6. The final plan is saved and presented with estimated costs and star ratings.

5.2 Testing

Software testing was conducted to ensure logical separations and functional stability across all edge cases. The primary challenge resided in validating the Add-on hotel logic against the immutable core logic.

5.2.1 Unit Testing

Individual components of the system were tested independently:

- **Tagging Logic Utility:** Verified that giving specific tags correctly returned the optimized subset of locations using mock local POI databases.
- **Rating Estimator Function:** Ingested pre-compiled positive and negative mock reviews ensuring the estimation formula mathematically altered the star coefficient accurately.
- **Price Engine Utility:** Ensured specific geographic locations strictly within Kerala's latitudinal boundaries computed pricing values within statistically reasonable thresholds.
- **Backend APIs:** Verified correct data handling and response generation for all core endpoints.

5.2.2 Integration Testing

Integration testing was performed to ensure smooth interaction between different modules. Tests were executed to guarantee that when the terminal point arrays of the generated itinerary were injected into the Hotel Add-On API, accurate radial searches were performed. Mocks were created to simulate users deciding to change a hotel for their second day. Successful integration tests verified that this specific change affected only accommodation records and did not trigger unintended state recalculations in the core itinerary.

5.2.3 Functional Testing

Functional testing was conducted based on user roles and system requirements:

- **End User:** Tested interest selection, itinerary generation, hotel browsing, and dynamic hotel switching.

- **Hotel Add-on:** Tested terminal location fetching, radial search accuracy, and continuity control logic.
- **Pricing Engine:** Tested tier classification for multiple Kerala locations against expected price brackets.

5.2.4 System and User Acceptance Testing

End-to-end scenarios involving complex travels, such as transitioning from coastal regions to mountainous regions, were tested. The UI successfully provided the "Change Hotel" and "Extend Stay" functionality properly linked only to their direct end-of-day routes. System latency strictly met the NFR requirement of 5 seconds per complex inference path.

5.2.5 Limitations in Testing

- Testing was conducted in a controlled development environment and may vary in real-world deployment.
- Performance under very high concurrent user load was not extensively stress-tested.
- Hotel data was sourced from a mock regional database; real-world accuracy will depend on the quality of live data integrations.

DC

5.3 Screenshots

This section visualizes how the underlying AI logic structures are represented to the end user, ensuring that extensive backend calculations manifest as intuitive interfaces.

Figure 5.1: Application Dashboard: Entry hub where users insert basic constraints, dates, and activate the smart tagging preference slider.

Figure 5.2: Generated Itinerary Interface: Showcasing the immutable core multi-destination route mapping generated by the AI planner.

Figure 5.3: Hotel Add-on Interface: The dynamic overlay showcasing terminal-location based accommodations, complete with the Estimated Star Rating and localized Kerala price generation.

Figure 5.4: Dynamic Stay Controls: Highlights the UI feature allowing travelers to either secure a new hotel targeted to the day's terminating hotspot or effortlessly continue their preceding stay.

Chapter 6

Conclusion

6.1 Future Scope

The current implementation of the Smart Itinerary Planner provides a strong foundation for AI-driven, modular travel planning. However, several enhancements can be introduced to improve its functionality and user experience in subsequent iterations:

- **Real-Time Traffic Adjustments:** Expanding the mapping APIs to pull live vehicular traffic metrics to restructure timeline predictions spontaneously if transit delays become severe.
- **Global Expansion of Cost Models:** Scaling the specialized Kerala-based localized pricing inference engine to encompass diverse national terrains and economic zones.
- **Booking Gateway Integrations:** Transitioning the Hotel add-on logic from conceptual estimation into direct commerce logic by integrating affiliate payment systems such as Stripe, mapped to live hotel inventory systems.
- **Mobile Application Support:** Developing a native mobile application to allow users to access and modify their itineraries on the go.
- **AI-Based Itinerary Refinement:** Integrating a conversational AI interface that allows users to modify itineraries using natural language commands.
- **Notification System:** Adding push notifications and email reminders for upcoming travel days and hotel check-in/check-out schedules.

6.2 Conclusion

The **Smart Itinerary Planner** successfully conceptualizes and prototypes a shift toward truly personalized, dynamic travel systems. By leveraging AI-based preference modeling and intelligent tagging algorithms, the project generates realistic, highly optimized multi-day travel plans that far exceed traditional manual compilation scopes.

The most notable systemic achievement is the successful isolation and implementation of the accommodation layer as a fully-featured, externalized add-on structure. This design decision fulfills a significant technological constraint: ensuring core geographical pathing logic is not adulterated by logistics scheduling frameworks. Through context-aware programming, checking end-of-day coordinates against regional databases properly limits transit fatigue and reduces unnecessary commute time for travelers.

Furthermore, integrating advanced evaluation matrices — specifically AI-inferred star ratings via text processing, combined closely with the specialized Kerala-based localized pricing computations — resolves a critical deficiency in mainstream, globally generalized travel applications.

By integrating these features, the Smart Itinerary Planner demonstrates an effective solution for AI-driven travel management. The system lays a scalable foundation that can be further enhanced with advanced features, real-time data integrations, and commercial booking capabilities in the future.

References

[1] Liu et al., "Satisfaction-Based Itinerary Planning," *Journal of Tourism Informatics*, 2024.

[2] Otaki and Baba, "Interaction-Based Itinerary Recommendation," *IEEE Access*, 2025.

[3] Espera et al., "+Tour: Smart Tourism with Multi-access Edge Computing," *IEEE Access*, 2025.

[4] Cui et al., "Intention-Aware Itinerary Recommendation," *IEEE Transactions on Neural Networks and Learning Systems*, 2025.

[5] Tang et al., "ITINERA: LLM-Based Itinerary Planning," *ACM Computing Surveys*, 2025.

[6] Morris and Petrova, "Learning-Based Itinerary Optimization," *Springer Lecture Notes in Computer Science*, 2024.